Received XX Month, XXXX; revised XX Month, XXXX; accepted XX Month, XXXX; Date of publication XX Month, XXXX; date of current version 14 October, 2026.

Digital Object Identifier 10.1109/XXXX.2026.1234567

Balancing Security and Usability: Adaptive CAPTCHA Orchestration via Reinforcement Learning

Anonymous Author(s)

[1]Affiliation Omitted for Double-Anonymized Review

ABSTRACT Risk-based bot detectors return a continuous score for each user rather than a decision, yet turning that score into an action (no challenge, a CAPTCHA of some difficulty, or a block) is still left to hand-written rules that trade security against usability. We study whether reinforcement learning (RL) can learn this mapping over an 11-level intervention ladder. A compact LSTM *Humanity Scorer* reads mouse dynamics (displacement, inter-event time, speed and acceleration) and outputs a calibrated bot probability every 10 s. With a running session score, challenge history and the number of active users, it forms the state of a DQN, PPO, Neural-LinUCB or Neural Thompson Sampling controller. Controllers are trained and evaluated in a stochastic simulator in which bots solve challenges with an item-response model and human pass rates and solve times come from published CAPTCHA user studies, so usability is *modelled, not measured*. On held-out recordings, PPO and DQN reach a Discrimination Index (humans kept % − bots through %) of 99.8 and 97.8 ($\pm$0.6 and $\pm$1.2, 95% CI over 5 seeds) against 95.5 for the best static threshold policy. When covert bots shift the scorer's calibration, frozen RL policies degrade faster than static thresholds. Training PPO online only once a label-free change detector fires keeps 99.8 without drift, alarms within 1–5 episodes of the attack, and reaches 85.1 and 82.0 at half and full shift, matching or exceeding online training with labels (83.3 and 82.6) and far above the best static policy (39.5 at full shift). RL can thus replace hand-tuned thresholds if the score is calibrated and monitored for drift; attacks that destroy the score's ranking remain out of reach.

INDEX TERMS Behavioural biometrics, bot detection, CAPTCHA, change detection, contextual bandits, mouse dynamics, online learning, probability calibration, reinforcement learning, web security.

I. INTRODUCTION

AUTOMATED traffic is now over half of all web requests, and 37% of 2024 traffic was attributed to malicious bots [1]. Bots scrape content, post spam [2] and stuff credentials [3]. The standard defence is the CAPTCHA [4], [5], but every challenge also costs legitimate users time, and some fail or leave: in a large-scale study of 318,000 CAPTCHAs, humans solved distorted-text schemes correctly only 70–98% of the time [6], and a recent study of modern CAPTCHAs found that a substantial share of participants abandoned a task when a challenge appeared [7]. Meanwhile, vision-language models solve many visual challenges [8], [9], and reCAPTCHA v2 has been broken with fine-tuned object detectors [10].

Risk-based services such as reCAPTCHA v3 therefore return a continuous score rather than a verdict [11], which moves the hard decision to the website operator: what should happen to a user whose score is 0.6? Three properties make this decision difficult. *One threshold does not fit every context*: a score that justifies frictionless browsing may be inadequate for a payment page [12], and fixed thresholds become stale as adversaries adapt [13], [14]. *Scores are not comparable across providers*: each detector has its own undisclosed calibration, so combining them becomes ad-hoc [15]. *The trade-off is sequential*: a mild first challenge can be escalated if a user still looks suspicious, and a policy that challenges early can relax later. Rule-based web application firewalls [16] encode these choices by hand, and the CAPTCHA schemes they trigger are each a single challenge type [17] rather than a graded response.

We ask whether an RL controller can learn the score-to-action mapping, and what a simulation-based evaluation can and cannot establish about it. Our contributions are:

i) **A graduated orchestration formulation.** Every 10 s of a session the controller chooses one of 11 levels (0: no challenge; 1–9: increasingly demanding challenges;

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

10: strongest fallback), from a behavioural score and session and server context (Section III).

ii) **A compact, calibrated behavioural scorer** (6,081 parameters), which matches or exceeds the mouse-movement detector of Iliou et al. [18] on their benchmark after fine-tuning (Sections III-C and IV-A).

iii) **A literature-grounded simulator**: bots solve challenges with a capability-dependent probability, and human pass rates and solve times come from published user studies [6], [7] (Section III-F).

iv) **An evaluation with confidence intervals** of DQN, PPO, Neural-LinUCB and Neural Thompson Sampling against static thresholds and a no-score ablation, on held-out recordings, under varying human and bot volumes (Section IV-B).

v) **Results that bound deployment**: rewards that need no labels, and frozen offline versus online training comparison when the scorer's calibration drifts (Section IV-D).

II. RELATED WORK

A. CAPTCHA Security and Usability

CAPTCHAs were introduced as hard AI problems [4] and have been surveyed extensively [5], [17]. Yan and El Ahmad analysed usability factors in CAPTCHA design [19]; Bursztein et al. measured human accuracy and solve time on 21 schemes [6]; and Searles et al. measured solve times, preferences and abandonment for modern challenges with 1,400 participants [7]. On the attack side, text CAPTCHAs have generic solvers [20], reCAPTCHA has been bypassed with browser cookies and history [14] and with image models [10], its v3 score has been gamed with RL [21], and multimodal models solve visual-reasoning challenges [8], [9].

B. Behavioural Biometrics

Mouse dynamics distinguish humans from bots [18], [22] and individuals from each other [23], [24]. Human motion follows Fitts' law with motor noise [25], while bots range from straight-line movers to Bézier-curve and learned trajectory generators [22], [26], [27]. Iliou et al. detected advanced web bots by fusing a web-log classifier with a mouse-movement classifier [18]; Acien et al. used touch and sensor data [28]. Recurrent networks such as the LSTM [29] have been used to detect bots from mouse-movement sequences [30].

C. Adaptive Challenges and RL

Aura-CAPTCHA uses RL and GANs to adapt challenge difficulty [31] but does not orchestrate a graded response ladder. Contextual bandits choose an action from features to maximise immediate reward [32], [33], while DQN [34] and PPO [35] optimise discounted return in a Markov decision process [36]. RL approaches have been applied to fraud and anomaly detection [37]. Labelled security data, including human/bot interaction data, remain scarce [38], which is why we report data sizes, splits and leakage controls in detail.

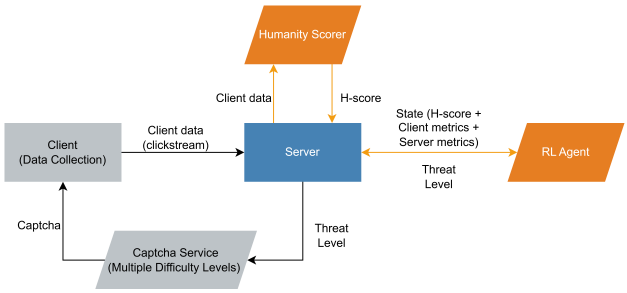


FIGURE 1. Modular architecture. Tiers 1–3 are implemented; the outcome of tier 4 is simulated.

III. METHODOLOGY

A. System Architecture

Fig. 1 shows the four tiers. *Tier 1* (client telemetry) collects cursor coordinates and timestamps in the browser and converts them into a platform-independent feature. *Tier 2* (Humanity Scorer) converts each 10 s chunk of telemetry into a bot probability; any scorer with a probabilistic output, including a commercial one, can be substituted. *Tier 3* (RL controller) maps the score and context to a level. *Tier 4* (fulfilment) routes each level to a concrete challenge; in this paper the outcome of each level is simulated (Section III-F).

B. Datasets

1) Our Recordings

A playground blog website (navigation, reading, form filling) recorded `mousemove` events (x, y, t) with a minimum gap of 10 ms; each session lasted about 120 s.

2) Bot Hierarchy

We implemented four levels of automated agents that perform the same tasks on the same site:

- *Naive bots* perform only the task (clicking a button, filling a form) with no attempt to look human; the cursor teleports directly to each target.
- *Human-like bots* move the cursor along straight lines instead of teleporting, but at constant speed, without realistic acceleration.
- *Mimic bots* follow non-straight Bézier curves with varying speed to emulate human motor characteristics [22], [26].
- *Fallible bots*, the most advanced, add human-like imperfections: occasional errors, hesitations and corrections such as overshooting a target and adjusting.

Human-like mouse movement lets advanced bots evade movement-based bot detectors [22], which is why the upper tiers imitate human motion rather than only completing the task. The dataset has 156 recordings (44 human, 112 bot; 42,913 movements); the human recordings come from 3 participants. The study followed established ethical guidelines for research with human participants: participants gave informed consent, and only anonymous cursor trajectories were stored, with no personally identifiable information retained.

3) Partition

Recordings are first split into a scorer block, an RL block and a held-out *evaluation block*. The evaluation block has 14 human and 36 bot recordings and is never used to fit or select any model, avoiding data snooping [39]. The scorer block alone (54 recordings) is too little data to train the scorer stably, so the scorer and RL blocks are combined into one training pool, and cross-fitting [40], [41] lets this pool serve both the scorer and the RL policy without leakage. The pool is split by recording into five parts; five scorers are each trained on four parts and score the RL-block recordings of the remaining part. The policy is trained on these out-of-fold scores, so no RL training session is scored by a model that saw it; otherwise the policy would learn from scores that are more confident than any it meets in deployment. For evaluation, one scorer is refitted on the whole pool and scores only the evaluation block.

4) Web Bot Detection Dataset

The web bot detection dataset of Iliou et al. [18] contains browser sessions on a Wikipedia-mirror site, with mouse movements and web logs, from humans, *moderate* bots (browser fingerprint and scripted movement) and *advanced* bots (browser fingerprint and human-like movement). Phase 1 has 50 human, 50 moderate and 50 advanced sessions and two benchmarks with fixed session-level splits: D1 (humans + moderate bots) and D2 (humans + advanced bots), each with 35+35 training and 15+15 test sessions. Phase 2 has sessions from 28 other users and new, separate bot sessions. We use Phase 1's moderate and advanced bots, through D1 and D2, for the benchmark comparison and finetuning (Section III-C), and only Phase 2's advanced bots, as covert bots, in the drift study (Section IV-D). Phase 2's moderate bots were excluded after a local check: our scorer failed against them and their ranking overlaps with humans, whereas the calibration-shift study needs bots whose scores move but whose ranking above humans survives. This was an accepted flaw of the Humanity Scorer in this experiment, since training data was not diverse enough to adapt to unseen data without finetuning.

5) Balabit Mouse Dynamics Challenge

The Balabit Mouse Dynamics Challenge [42] contains remote-desktop mouse sessions of 10 users. It contains only humans, so we use it only as a population of unseen humans.

6) Preparation of External Data

- *Cleaning (all external data):* Events with pixel coordinates outside $[0, 10^4]$ are dropped, and so is any event less than 10 ms after the previous kept one, resampling every source to our logger's rate.
- *Benchmark (Phase 1 D1/D2):* Each session is cut into consecutive 10 s chunks; chunks with fewer than two events are dropped. Sessions and splits are the dataset's own.
- *Drift study (Phase 2 advanced bots, Balabit):* Only the first 120 s of a session is kept, and a session needs at least 30 s and 20 events to qualify. This leaves 120 advanced-bot sessions. Balabit is capped at 30 qualifying sessions (drawn with fixed random seed) per user, so that no user dominates: 300 sessions from 10 users.
- *Splitting for online training and evaluation:* Each population is split once: the advanced-bot sessions into two halves of 60 by a fixed-seed shuffle, and Balabit by user, 5 users (150 sessions) per half, so evaluation humans are people never seen in training. One half supplies the covert bots or bot-like humans mixed into online training traffic, the other only into evaluation traffic.

C. Humanity Scorer

1) Input

A chunk is converted to per-event features $[\Delta x, \Delta y, \Delta t, v, a]$: displacement, inter-event time, speed $v = \|(\Delta x, \Delta y)\|/\Delta t$ and acceleration $a = \Delta v/\Delta t$, both finite-difference estimates between consecutive events. Absolute screen position is not used, because it encodes page layout rather than behaviour. Features are standardised with training statistics and cut into windows of 32 events; the last window is padded and a sixth binary channel marks padded steps (Fig. 2).

2) Model

One LSTM layer [29] with 32 units reads the window. Given the input x_t, previous hidden state h_{t-1} and cell state c_{t-1}, it computes

$$\begin{aligned} f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f), \\ i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i), \\ o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o), \\ \tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c), \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \qquad h_t = o_t \odot \tanh(c_t), \end{aligned} \tag{1}$$

where the forget, input and output gates f_t, i_t, o_t decide what the cell keeps, adds and exposes, and $\odot$ is the element-wise product. The final hidden state h_{32} passes through a 32-unit ReLU layer and a single linear output, the logit z (6,081 parameters in total). Training uses binary cross-entropy with class-balanced weights, Adam (learning rate 3×10^{-3}), dropout 0.25 and recurrent dropout 0.2, with early stopping on a validation split of recordings.

3) Temperature Calibration

Since the RL state and the rewards treat the score as a probability, the network's output is calibrated after training by temperature scaling [43]. For a window with logit z, the calibrated probability is $\sigma(z/T)$, where the single parameter $T > 0$ is chosen to minimise the log-loss on the validation split and the network weights are left unchanged; $T > 1$ softens over-confident outputs and $T < 1$ sharpens under-confident ones, while the ranking of windows, and hence the AUC, is unaffected. The scorer used throughout has

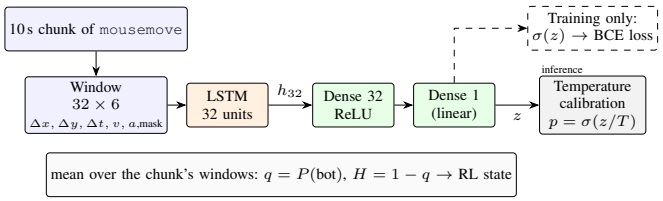


FIGURE 2. Humanity Scorer Architecture.

$T = 1.14$. The chunk score is the mean of its windows' calibrated probabilities, $q = P(\text{bot})$, and the Humanity Score is $H = 1-q$. Section IV-A reports its calibration on held-out recordings.

4) Fine-Tuning on the Web Bot Benchmark

We use the benchmark's own split of D1 and D2 into 70 training sessions (35 human, 35 bot) and 30 test sessions (15 and 15). Each session is cut into chunks as in Section III-B, using all its `mousemove` events (no 120 s limit). Each chunk becomes 32-event windows of $[\Delta x, \Delta y, \Delta t, v, a]$ plus the padding mask, standardised with the pre-trained scorer's statistics, as in Section III-C. This gives 1,681 training and 719 test chunks for D1 and 2,186 and 942 for D2. The model is our final refitted scorer fine-tuned on the training sessions: 20% of the training sessions of each class (7 human and 7 bot) are held out for early stopping (patience 10, till 100 epochs) and for calibrating the temperature, while the rest are used for training with Adam (learning rate 10^{-4}), batch size 64 and class-balanced weights. A chunk's score is the mean of its windows' calibrated probabilities, and a session's score is the mean of its chunk scores, thresholded at 0.5. We repeat this with five seeds, which change the validation split and the randomness of training (batch order and dropout), and report the mean.

5) Design Rationale

The architecture and input representation were selected by 5-fold cross-validation grouped by session, over LSTM sizes from 8 to 64 units, one or two layers, and inputs ranging from raw coordinates to the kinematic features above. Small models generalised best to unseen data splits. Data augmentation (Gaussian or per-movement jitter) did not improve this model and hence is not used.

6) Static Baselines

Four untrained policies map the score straight to a level. The *static threshold* shows level 10 when $q > 0.5$, and the *multi-threshold* shows level $\lfloor 10q \rfloor$. The *NP threshold* sets t_α by the Neyman–Pearson criterion [44], [45], so that at most a fraction α of the scorer block's human sessions would ever be challenged, and shows level 10 above it; the *NP multi-threshold* maps ten equal bands of $(t_\alpha, 1]$ to levels 1–10. We use $\alpha = 0.1$ ($t = 0.685$), the best of the values tried in Section IV-D.

D. RL Formulation

Each user session is an episode of up to 12 decision steps, one per 10 s chunk. Many users are active at once, and the controller acts for each active user at each step.

1) State

Five features in $[0, 1]$: (1) the current chunk score q_t; (2) the running mean of q over the session; (3) the number of challenges the user has passed, $\tanh(n/5)$; (4) the last level shown, divided by 10; and (5) the number of active users, scaled. Features (3)–(5) need no scorer.

2) Actions

Level $T \in \{0, \dots, 10\}$. Level 0 shows nothing; levels 1–2 are invisible to the user (passive score, honeypot); levels 3–9 are visible challenges of increasing difficulty; level 10 is the strongest level, an audio fallback that humans pass about half the time and only the most capable bots solve; it is not a hard block, so no level removes a user with certainty (Table 1).

3) Reward (Training Only)

For a bot of capability B_s: if it is stopped, $r = R_{\max} - c_o \max(0, T - B_s)$, which discourages stronger levels than needed; if it passes, $r = R_{\text{leak}} - c_u \max(0, B_s - T)$. For a human: $r = R_{\max}$ at $T = 0$; $r = R_{\max}/2 - c_f\, s_T$ after passing a challenge that took s_T seconds; and $r = R_{\text{FP}} - c_f\, s_T$ if the human fails it (a false positive). Friction is priced per measured second, so the human cost of a level follows the solve times of Table 1. This reward needs the user's true class, which the simulator knows and a deployed system does not.

4) Worked Example

A fallible bot of capability $B_s = 4$ produces a second chunk with $q = 0.82$; its session mean is 0.75, it has passed no challenges, its last level was 0 and 400 users are active. The state is $(0.82, 0.75, 0, 0, 0.67)$. If the policy chooses $T = 6$ (image grid), the bot passes with probability $\sigma(1.5(4.5 - 6)) = 0.095$. If it is stopped, the session ends and $r = 50 - 2.5 \cdot 2 = 45$; if it passes, $r = -150$ and its next state records one passed challenge and last level 0.6. A human shown the same level passes with probability 0.86 in a median 9.7 s, giving $r = 25 - 2 \cdot 9.7 = 5.6$, or fails, giving $r = -100 - 19.4$. A trained DQN typically shows nothing to a user with $q < 0.2$ and escalates through levels 3–7 as q and the running mean rise.

5) Online Rewards

The reward above needs the class. We also train with four rewards that need at most a small labelled sample. Let o_t be the observable outcome of step t (no challenge, passed, failed and continued, failed and left), $\bar{r}_H(T, o)$ the human reward at expected friction, and $r_B(b, T, o)$ the reward of a bot of capability b, with prior $\varrho(b)$ uniform over $0, \dots, 9$. The *score-only* reward reads the chunk score as $P(\text{bot})$:

$$\hat{r}_t^{\text{score}} = (1 - q_t)\, \bar{r}_H(T_t, o_t) + q_t \sum_b \varrho(b)\, r_B(b, T_t, o_t). \quad (2)$$

The *posterior* reward replaces q_t by the posterior over class and capability given the score and the session's outcomes so far:

$$\begin{aligned} w_t(H) &\propto (1-\hat{\pi})\textstyle\prod_{j\le t} P_H(o_j \mid T_j),\\ w_t(B,b) &\propto \hat{\pi}\,\Lambda_t\,\varrho(b)\textstyle\prod_{j\le t} P_B(o_j \mid T_j, b),\\ \hat{r}_t^{\text{post}} &= w_t(H)\,\bar{r}_H(T_t,o_t)\\ &\quad + \textstyle\sum_b w_t(B,b)\,r_B(b,T_t,o_t), \end{aligned} \tag{3}$$

normalised to sum to one. P_H, P_B are the simulator's outcome probabilities, so this reward is an upper bound for the idea; Λ_t is the score likelihood ratio from a Platt calibration [46] fitted on the scorer block; and $\hat{\pi}$ is the episode's bot share, estimated by EM from the users' first-step ratios [47].

The two *audit* rewards recalibrate the score on current traffic from a random 2% of users per episode. With labels (from review or confirmed abuse), ten score bins $k(q) = \min(\lfloor 10q \rfloor, 9)$ with centres c_k keep decayed counts of audited steps and bots, a variant of histogram-binning recalibration [48]:

$$\begin{aligned} n_k &\leftarrow 0.97\,n_k + |\mathcal{A}_{e,k}|,\\ m_k &\leftarrow 0.97\,m_k + \textstyle\sum_{i\in\mathcal{A}_{e,k}} y_i,\\ \hat{p}_k &= \frac{m_k + 2c_k}{n_k + 2}, \qquad \hat{r}_t^{\text{audit}} = \hat{r}_t^{\text{score}}\big|_{q_t \to \hat{p}_{k(q_t)}}, \end{aligned} \tag{4}$$

where $\mathcal{A}_{e,k}$ holds the audited steps of episode e in bin k and $y_i = 1$ for a bot; audited users keep the label-based reward. With a level-9 challenge as the audit, shown before the policy acts and kept on at evaluation, only pass or fail is observed: m_k counts failures, and the bot share follows from the known fail rates $f_H = 0.144$, $f_B = 0.887$ by a smoothed adjusted classify-and-count [49],

$$\begin{aligned} \hat{f}_k &= \frac{m_k + 2\,[c_k f_B + (1-c_k) f_H]}{n_k + 2},\\ \hat{p}_k &= \mathrm{clip}_{[0,1]}\Big(\frac{\hat{f}_k - f_H}{f_B - f_H}\Big), \end{aligned} \tag{5}$$

which then replaces q_t in (2) for every user.

E. Learning Algorithms

1) DQN

DQN [34] approximates the optimal action-value function $Q^*(s,a)$ with a 128–64 ReLU network $Q(s,a;\theta)$ that outputs one value per level. Transitions (s,a,r,s') are stored in a replay memory $\mathcal{D}$, and the network minimises the temporal-difference loss

$$\begin{aligned} \mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\Big[\ell_\delta\big(r + \gamma \max_{a'} Q(s',a';\theta^-)\\ - Q(s,a;\theta)\big)\Big], \end{aligned} \tag{6}$$

where ℓ_δ is the Huber loss and θ^- are the parameters of a target network copied from θ every 200 steps. Actions are chosen ϵ-greedily: a random level with probability ϵ (decayed from 1.0 to 0.01), otherwise $\arg\max_a Q(s,a;\theta)$. The replay memory is split into a human and a bot buffer (10,000 transitions each), and every mini-batch draws half from each; this balancing uses the true class, as the reward does.

2) PPO

PPO [35] learns a stochastic policy $\pi_\theta(a \mid s)$ (11 logits) and a value function from the same 128–64 body. It maximises the clipped surrogate objective

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\big[\min\big(\kappa_t \hat{A}_t,\ \mathrm{clip}(\kappa_t, 1-\epsilon_c, 1+\epsilon_c)\hat{A}_t\big)\big], \tag{7}$$

where $\kappa_t = \pi_\theta(a_t \mid s_t)/\pi_{\theta_{\text{old}}}(a_t \mid s_t)$ is the probability ratio, $\hat{A}_t$ the generalised advantage estimate and $\epsilon_c = 0.2$; clipping stops a single update from moving the policy too far. Value-function and entropy terms are added with weights 0.5 and 0.01, and the policy is updated once per simulated episode.

3) Neural-LinUCB and Neural Thompson Sampling

Neural-LinUCB and Neural Thompson Sampling [32], [33], [50] treat each decision as a contextual bandit. An MLP maps the state to an embedding $\phi(s)$, and the expected reward of level a is modelled as linear in it, $\mathbb{E}[r \mid s,a] = \phi(s)^\top \theta_a$. For each level the bandit keeps the ridge statistics

$$A_a = \lambda I + \sum_{i:a_i=a} \phi(s_i)\phi(s_i)^\top, \qquad b_a = \sum_{i:a_i=a} r_i\,\phi(s_i), \tag{8}$$

so that $\hat{\theta}_a = A_a^{-1} b_a$ is the ridge estimate ($\lambda = 1$), and the network weights are trained to reduce the squared error between observed and predicted rewards. LinUCB is optimistic under uncertainty: it picks

$$a_t = \arg\max_a\ \phi(s_t)^\top \hat{\theta}_a + \alpha\sqrt{\phi(s_t)^\top A_a^{-1}\phi(s_t)}, \quad \alpha = 1. \tag{9}$$

Thompson Sampling instead keeps a posterior over each θ_a, draws $\tilde{\theta}_a$ from it and picks $\arg\max_a \phi(s_t)^\top \tilde{\theta}_a$. With one component the sample is $\tilde{\theta}_a \sim \mathcal{N}(\hat{\theta}_a, \alpha^2 A_a^{-1})$, with the covariance either diagonal or full. A single linear model per level cannot express that the same level pays very differently for a human and for a bot, so we also allowed a mixture of K components per level, in the spirit of mixture-prior Thompson Sampling [51], fitted without labels on (embedding, reward) pairs: each component k has its own ridge statistics and a Gaussian density over the embedding, and a context-dependent gate $\pi_k(\phi) \propto \pi_k\,\mathcal{N}(\phi; m_k, v_k)$ weights the components' sampled scores, $\text{score}(a) = \sum_k \pi_k(\phi)\,\phi^\top \tilde{\theta}_{a,k}$. With $K = 1$ this reduces exactly to Gaussian Thompson Sampling.

Four posteriors were compared in the architecture search: a single Gaussian with diagonal covariance, a single Gaussian with full covariance, and mixtures of two and three components (MoG2, MoG3), each combined with five embedding networks. This gives 20 candidates (4 posteriors $\times$ 5 networks). Each candidate was trained with 5 seeds and scored by 3-fold cross-validation on the RL block. LinUCB, whose rule has no posterior to choose, was searched over the same five embedding networks with its standard single-Gaussian statistics. Since ϕ changes whenever the network is updated, statistics accumulated under earlier embeddings

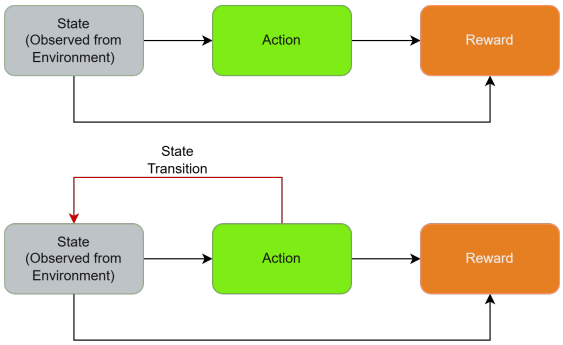


FIGURE 3. **Contextual bandit (top) and full RL (bottom) interaction loops.**

become inconsistent; at the end of training we therefore recompute every A_a and b_a from the stored transitions in the final embedding space.

Fig. 3 contrasts the two families: bandits treat each decision independently, while DQN and PPO propagate value across the steps of a session. Table 2 lists the final configuration of every component.

F. Simulated Environment

1) Traffic

Each training episode draws 50–250 humans uniformly and, independently, one bot volume from $\{0, 20, 100, 200, 500, 1000\}$, so the number of active users does not reveal the number of bots. Each simulated user replays the mouse data of a session recording from the RL block, and each bot draws a capability B_s uniformly from $\{0, \ldots, 9\}$. The trained policies are then evaluated, frozen, in two traffic settings, fixed and varying (Section III-G).

2) Challenge Outcomes

Table 1 lists the 11 threat levels.

Bots: A bot's capability $B_s \in \{0, \ldots, 9\}$ sets the hardest level it solves reliably ($B_s - 1$, pass probability 0.90); at level B_s itself it passes about two times in three. It passes level $T \geq 1$ with the two-parameter logistic item-response (2PL) probability $\sigma(\alpha(B_s + 0.5 - T))$, the standard model for an agent of latent ability attempting an item of given difficulty [52], [53], with discrimination $\alpha = 1.5$, a value in the "high" range (1.35–1.69) of the usual classification of item discriminations [54], chosen because each level of the ladder is meant to separate bot capabilities sharply while keeping outcomes probabilistic. No measurement fixes α, so we treat it as a design choice and test its sensitivity over $\alpha \in \{0.5, 1, 1.5, 3, 6, 12\}$ (Section IV-C); level 0 shows nothing and is always passed. Strong bots occasionally fail easy challenges and weak bots occasionally pass hard ones. No published study reports solve rates for a common set of bots across all eleven mechanisms, so the per-level bot rates are model outputs rather than measurements; the ordering of the levels follows the attack literature in Table 1 (passive scores and checkboxes are defeated by browser-driving bots, distorted text by generic solvers, image grids by object detectors, while reasoning games are solved only partially by recent multimodal agents).

Humans: A human passes each attempt with the measured pass probability of that challenge type and takes its median solve time; a human who fails may retry once and is lost after a second failure. Even if capable of passing, a human leaves the site with probability $1 - e^{-s_T/\tau}$, $\tau = 45$ s, where s_T is the time spent. This models human abandonment from friction. We found no study that measures such leaving, so this term is a modelling assumption and we report its sensitivity (Section IV-C).

G. Evaluation Protocol and Metrics

Each policy is trained on the RL block and evaluated, frozen, on simulated traffic built from the evaluation block in two settings: *fixed* traffic of 100 humans with 20, 100, 200, 500 or 1000 bots, and *varying* traffic in which the number of humans is drawn from 50–250 and bots make up 5–75% of all users (so 3–750 bots). Each setting is run with 50 evaluation seeds for each of 5 training seeds.

Let H_{tot} and B_{tot} be the numbers of humans and bots in a run, and H_{surv} and B_{surv} the numbers that complete their session. With human survival $S_H = H_{\text{surv}}/H_{\text{tot}}$, bot survival $S_B = B_{\text{surv}}/B_{\text{tot}}$ and bot kill rate $K_B = 1 - S_B$:

- **Discrimination Index**, DI $= 100(S_H - S_B)$: the margin between the share of humans kept and bots let through. It is our primary metric.
- **Balanced Orchestration Score**, BOS $= 2S_H K_B/(S_H + K_B)$: the harmonic mean of humans kept and bots stopped, which is low whenever either fails [56].
- **System-Purity F1**, SP-F1 $= 2PS_H/(P + S_H)$ with $P = H_{\text{surv}}/(H_{\text{surv}} + B_{\text{surv}})$: the purity of the traffic that gets through; it depends on the bot-to-human ratio.
- **Scale-Invariant F1**, SI-F1 $= 2P_n S_H/(P_n + S_H)$ with $P_n = S_H/(S_H + S_B)$: a version of SP-F1 that weights 1% of humans equally with 1% of bots, so it does not depend on traffic volume.

We also report mean friction (seconds of challenge time per human). Bot survival is undefined without bots, so every run has at least one bot. Uncertainty is reported across training seeds (each seed averaged over its evaluation runs) as a 95% t-interval.

IV. RESULTS

A. Humanity Scorer

1) Our Recordings

The five cross-fitted scorers reach validation AUC 0.976–0.988 and AUC 0.962–0.984 on the recordings held out from each. On the evaluation block the refitted scorer classifies 96.9% of 262 chunks correctly (AUC 0.992; 154 of 160 bot chunks and 100 of 102 human chunks), and all 50 sessions when chunk scores are averaged per session. Its chunk-level Brier score is 0.032 and its expected calibration error (10 bins) is 0.053. Temperature scaling, fitted on the validation split, does not reduce calibration error on this block (0.045

TABLE 1. Action Ladder of the Simulated Environment. Human Pass Probability and Median Time per Attempt; Bot Pass Probability $\sigma(1.5(B_s+0.5-T))$ for $T \geq 1$, Shown for Three Capabilities (Every Other Capability Follows from the Gap $B_s - T$, Section III-F)

		Human			Bot			
T	**Mechanism**	P(pass)	Median s	Source	B_s=2	B_s=5	B_s=9	Evidence that automated solvers defeat this type
0	No challenge	1.000	0	—	1.000	1.000	1.000	nothing is shown
1	Passive risk score	0.999	0	assumed (invisible to the user)	0.905	0.999	1.000	reCAPTCHA v3 score gamed by an RL agent [21]
2	Honeypot / JavaScript challenge	0.995	0.5	assumed (invisible to the user)	0.679	0.995	1.000	defeated by any browser-driving bot [18]
3	Checkbox click	0.980	3.7	time [7]; pass rate assumed	0.321	0.977	1.000	reCAPTCHA checkbox passed with browser history and cookies [14]
4	Easy distorted text	0.980	6.8	[6] (authorize.net)	0.095	0.905	1.000	generic text solver broke all 8 schemes tested [20]
5	Standard distorted text	0.930	7.3	[6] (eBay)	0.023	0.679	0.999	generic text solver [20]
6	Image grid selection	0.860	9.7	[6] (Google)	0.005	0.321	0.995	a fine-tuned detector solves 100% of reCAPTCHA v2 image-grid sessions [10]
7	Hard distorted text	0.750	11.9	[6] (reCAPTCHA)	0.001	0.095	0.977	harder schemes resist the generic solver more often [20]
8	Very hard text / multi-round	0.700	13.0	[6] (mail.ru)	0.000	0.023	0.905	several rounds must all be solved (assumed)
9	Interactive game	0.620	25.0	time [7] (18–42 s); pass rate assumed	0.000	0.005	0.679	only recent VLM agents solve reasoning challenges, partially [8], [9]
10	Audio fallback (strongest level)	0.520	30.0	pass rate [6] (audio); time assumed	0.000	0.001	0.321	strongest level; passed only by the most capable bots (assumed)

TABLE 2. Final Configuration. Basis: L = Published Study; CV = Selected by Cross-Validation on the RL Block (Never on the Evaluation Block); D = Design Choice, with the Sensitivity Range Tested in Section IV-C

Component	**Setting**	**Basis**
Episode	One decision per 10 s chunk, up to 12 per session; 50–250 humans and 0–1000 bots per episode, drawn independently	D (recordings last ≈120 s)
Bots	Capability $B_s \sim \mathcal{U}\{0,\ldots,9\}$; pass probability $\sigma(\alpha(B_s+0.5-T))$ with $\alpha = 1.5$	D; α from 0.5 to 12 tested
Humans	Pass rate and time per level (Table 1); one retry; abandonment with probability $1-e^{-s_T/45\,\mathrm{s}}$	L; D for leaving (τ 22–80 s and no leaving tested)
Reward	$R_{\max} = 50$ (human at level 0, stopped bot), $R_{\text{leak}} = -150$ (bot passes), $R_{\text{FP}} = -100$ (human fails); overkill $c_o = 2.5$ and underestimation $c_u = 5$ per level; friction $c_f = 2$ per second	D
Humanity Scorer	LSTM(32) → Dense(32, ReLU) → sigmoid; 32-event windows of $[\Delta x, \Delta y, \Delta t, v, a]$ + mask; Adam 3×10^{-3}, batch 64, dropout 0.25/0.2, early stopping; temperature scaling	CV
DQN	128–64 ReLU, 9,739 parameters; Huber loss; $\gamma = 0.95$; Adam 5×10^{-4}; batch 64 (32 human + 32 bot transitions); buffers of 10,000 per class; ϵ: 1.0 → 0.01 (decay 0.9995); target sync every 200 steps	CV (body); standard [34]
PPO	Shared 128–64 ReLU body; clip 0.2; GAE $\lambda = 0.95$; $\gamma = 0.95$; 10 epochs of minibatch 64 per episode; Adam 3×10^{-4}; entropy 0.01; value 0.5	CV (body); standard [35], [55]
Neural-LinUCB	MLP 64 → 32-d embedding; $\alpha = 1$; ridge $\lambda = 1$	CV (body); standard [32], [50]
Neural Thompson Sampling	MLP 128–64 → 64-d embedding; 3-component mixture-of-Gaussians posterior	CV [50], [51]
Training and evaluation	200 training episodes; 5 training seeds; 50 evaluation seeds per traffic setting	D

before scaling), so we treat the score as approximately but not exactly calibrated.

2) Web Bot Detection Benchmark

Table 3 compares our fine-tuned scorer (Section III-C), which uses mouse movements only, with the two detectors of Iliou et al. [18]: their mouse-movement module and their fusion of mouse movements and web logs.

The fine-tuned scorer classifies every test session correctly on both benchmarks in all five seeds. On mouse movements alone it matches Iliou et al.'s mouse module on D1 and exceeds it on D2 (accuracy 1.000 vs 0.93), the benchmark whose bots imitate human movement, and it equals their fusion with web logs. Per 10 s chunk, accuracy is 0.996 (D1) and 0.995 (D2), with AUC 0.999.

B. Reinforcement Learning

Table 4 reports every policy on the evaluation block. PPO and DQN keep almost every human and let no bot through: PPO reaches DI 99.8 (worst seed 99.0) with 0.1 s of challenge time per human, and DQN 97.8 (worst seed 97.0), losing a few humans only at 500 and 1000 bots. Both exceed every static policy. The NP threshold stops every bot but loses 4.5% of humans (DI 95.5, 3.0 s per human); the NP multi-threshold keeps slightly more humans (96.2%) but lets 1.9% of bots through (DI 94.4). Their fixed threshold variants perform much below them. Removing the Humanity Score reduces DQN to DI 16.8, letting 45% of bots through.

The bandits are less stable: Thompson Sampling reaches DI 92.1 (worst seed 81.7) and LinUCB 64.7 (worst seed

TABLE 3. Web Bot Detection Dataset [18]: D1 (Humans vs Moderate Bots) and D2 (Humans vs Advanced Bots), Official Test Sessions. Iliou et al.: Values Reported in [18] (Session Level). Ours: Fine-Tuned Humanity Scorer, Mean ± SD over 5 Seeds

	D1: humans vs moderate bots								D2: humans vs advanced bots							
		Bot class			Human class					Bot class			Human class			
Detector (level)	Acc.	P	R	F1	P	R	F1	AUC	Acc.	P	R	F1	P	R	F1	AUC
Iliou et al., mouse module (session)	1.00	1.00	1.00	1.00	1.00	1.00	1.00	–	0.93	0.88	1.00	0.94	1.00	0.87	0.93	–
Iliou et al., mouse + web-log fusion (session)	1.00	1.00	1.00	1.00	1.00	1.00	1.00	–	1.00	1.00	1.00	1.00	1.00	1.00	1.00	–
Ours, fine-tuned, per session[a]	**1.000**	1.000	1.000	1.000	1.000	1.000	1.000	1.000	**1.000**	1.000	1.000	1.000	1.000	1.000	1.000	1.000
Ours, fine-tuned, per 10 s chunk[b]	0.996	0.988	1.000	0.994	1.000	0.994	0.997	0.999	0.995	0.997	0.993	0.995	0.993	0.997	0.995	0.999

[a] One decision per test session: the mean of the session's chunk scores, thresholded at 0.5; 30 test sessions in each benchmark (15 human, 15 bot), the unit Iliou et al. report.
[b] One decision per 10 s chunk of a test session, the unit our controller acts on; 719 test chunks in D1 and 942 in D2. Seed-to-seed SD ≤0.001 for every entry.

TABLE 4. Held-Out Evaluation. Fixed Traffic: 100 Humans and 20–1000 Bots; Varying Traffic: 50–250 Humans with Bots Making Up 5–75% of Users. Means over 50 Evaluation Seeds per Setting; ± Is the 95% CI over 5 Training Seeds. Static Policies Have No Training Seed.

	DI by bot volume (fixed)					Fixed traffic, all volumes							Varying traffic
Policy	20	100	200	500	1000	**DI**	Worst seed	BOS	SP-F1	SI-F1	Humans kept	Friction (s)	**DI**
PPO	**100.0**	**100.0**	**100.0**	**100.0**	**99.0**	**99.8** ± 0.6	**99.0**	**0.999**	**0.998**	**0.999**	**0.998**	**0.1**	**99.96** ± 0.07
DQN	**100.0**	**100.0**	**100.0**	95.4	93.8	97.8 ± 1.2	97.0	0.989	0.989	0.989	0.978	1.3	99.4 ± 0.3
Thompson Sampling (MoG3)	88.2	96.1	94.9	93.5	87.6	92.1 ± 10.1	81.7	0.953	0.949	0.953	0.925	4.9	92.3 ± 11.5
TS, MoG2	79.2	88.7	93.6	92.5	81.4	87.1 ± 12.7	74.5	0.923	0.921	0.930	0.911	5.8	88.4 ± 13.8
TS, Gaussian (full)	67.8	78.4	81.9	82.8	79.5	78.1 ± 24.4	45.2	0.860	0.860	0.860	0.781	12.3	77.0 ± 24.3
TS, Gaussian (diag.)	68.2	77.7	80.5	82.0	79.6	77.6 ± 24.6	44.8	0.857	0.857	0.857	0.776	12.6	76.6 ± 25.1
LinUCB	67.1	67.0	65.7	64.9	58.9	64.7 ± 28.3	44.9	0.768	0.741	0.768	0.665	18.1	62.2 ± 33.4
Static NP threshold ($\alpha = 0.1$)	95.7	95.7	95.6	95.3	95.0	95.5	—	0.977	0.977	0.977	0.955	3.0	95.2
Static NP multi-threshold ($\alpha = 0.1$)	94.8	94.1	94.6	94.4	94.1	94.4	—	0.972	0.945	0.972	0.962	2.3	93.9
Static threshold	90.6	91.0	90.9	90.7	90.5	90.7	—	0.951	0.951	0.951	0.907	6.2	90.3
Static multi-threshold	77.0	77.4	77.9	78.5	77.8	77.7	—	0.875	0.867	0.875	0.783	10.7	78.0
DQN without Humanity Score	5.1	16.7	23.7	21.4	17.1	16.8 ± 7.5	9.3	0.394	0.487	0.555	0.614	20.0	16.2 ± 7.0

44.9). Refitted on held-out recordings, the Thompson posteriors reproduce the ordering of the architecture search (single Gaussian 77.6 diagonal and 78.1 full, two components 87.1, three 92.1), so the gain comes from the number of components, not the covariance. Thompson Sampling (MoG3) with 3 components performs the best among them. Optimising only the immediate reward, the bandits cannot plan an escalation over the session as DQN and PPO do. Under varying traffic every policy scores the same or slightly higher (PPO 99.96, DQN 99.4). The evaluation block is small and well separated by the scorer, so these figures are close to a ceiling; Section IV-D tests the policies when the score is less reliable.

C. Sensitivity to Simulator Assumptions

1) Simulator Constants

We re-evaluated the trained policies (without retraining) under bot-solve slopes $\alpha \in \{0.5, 1, 1.5, 3, 6, 12\}$ and leave-after-pass constants $\tau \in \{22, 45, 80\}$ s or no leaving. RL policies are stable across α, while the multi-threshold policy drops to 69.6 at $\alpha = 12$. Human abandonment matters most for policies that challenge humans often: as τ grows from 22 s to no leaving, the multi-threshold policy rises from 66.2 to 97.9 and overtakes the static threshold (87.8 to 96.8). PPO and DQN remain the best policies at every τ.

2) Reward Constants

DQN was retrained (5 seeds each) under nine alternative reward settings: leak penalty $R_{\text{leak}} \in \{-50, -300\}$, false-positive penalty $R_{\text{FP}} \in \{-50, -200\}$, friction $c_f \in \{1, 4\}$ per second, overkill $c_o = 1$ and underestimation $c_u \in \{2.5, 10\}$. Eight settings give DI 97.5–98.7 (default 97.7), with no bot getting through in any of them. The exception is the mildest false-positive penalty ($R_{\text{FP}} = -50$): losing a human then costs little more than a challenge, and one of its five seeds kept only 70% of humans (DI 70.4; mean 92.1). Every setting stays above the static threshold (90.3 on the same evaluation seeds) on average.

3) Population Input

Freezing DQN's active-user feature at 25, 100 or 400 regardless of the real traffic leaves bot survival at zero (DI 100.0, against 96.6 unfrozen on the same grid of 25–400 humans and 20–1000 bots). Freezing it at 1400, above anything seen in training, makes the policy challenge more and keep fewer humans (87%). The population count therefore controls how cautious the policy is, not whether it stops bots; detection comes from the score.

D. Covert Bots: Offline vs Online Training

New bot types are the main threat a deployed scorer faces: bots that imitate human movement more closely than anything in the scorer's training data. We test two versions of

this threat. In the first, the scorer's *calibration* breaks while its ranking survives; in the second, the attacker destroys the ranking as well.

1) Covert Bots That Shift Calibration

A fraction ρ of the evaluation bots is replaced by Phase 2 advanced web bots [18], whose movement imitates humans; training uses the other half of that population. Without fine-tuning, our scorer gives these bots a mean $P(\text{bot})$ of only 0.39, so as ρ goes from 0 to 1 the bots' calibration error grows from 0.07 to 0.61, while session-level AUC stays ≥ 0.99: the bots still score above the humans, but far below what the policies learned to expect. We compare policies trained offline and evaluated frozen with DQN and PPO trained online for 300 episodes, with the covert bots present from episode 100, against the same protocol without them (control). This online training result for PPO under drift is shown in Table 5.

Frozen policies (Fig. 4, left) all degrade as more bots become covert, and the RL policies degrade *further* than static thresholds, since they learned to trust the score more; DQN without the score stays flat at about 16, as expected for a negative control. *Online training* (middle and right) fixes this. At $\rho = 1$ PPO trained with the score-only reward keeps DI at 82.7, against 3.8 frozen and 39.5 for the best static policy. Online updates move the policy's operating point as the scores move, which no fixed threshold can do. The posterior reward is the exception: its EM estimate of the bot share treats covert bots as humans, so online training with it lets essentially every bot through at $\rho = 1$ (DI ≈ 0), *below* its own control. A reward that estimates the bot population from the score inherits the scorer's blind spot.

Trade-off between humans lost and bots stopped: Fig. 5 sweeps each method's trade-off parameter: the NP budget α for the static policy and the false-positive penalty $R_{\text{FP}} \in \{-25, \ldots, -400\}$ for DQN and PPO, trained offline (frozen) or online through the covert bots with the score-only reward (3 seeds each). Without drift, offline PPO stops every bot while losing under 1% of humans; the best static threshold needs 4.4%. With covert bots the frozen policies stop only 5–14% of bots whatever the penalty, while online PPO stops 99–100% at 16–18% of humans lost and online DQN 64–72% at 5–10%; static policies stop all covert bots only at 27% of humans lost. The penalty barely moves the frozen policies: their failure under drift comes from trusting the score, which the penalty does not change.

2) Recalibrating the Reward with a Small Audit Sample

A middle ground between the score-only reward and labelling every user is to audit a random 2% of users and use their labels to recalibrate the score on the *current* traffic (Eqn. (4), Section III-D); because the audited sample is random, the estimate stays unbiased whatever the drift does to the score. Table 5 compares both audit rewards with the other rewards for PPO.

TABLE 5. PPO Trained Online Through the Shift with Different Rewards. Online Training: PPO Learns from Scratch for 300 Episodes with the Given Reward, the Shift Starting at Episode 100; DI on the Evaluation Block at the Shifted Point, Mean ± SD over 5 Training Seeds, with Friction (Seconds of Challenge per Human) in Parentheses; No-Shift Column: the Same Training without the Shift

	No	Covert bots		Bot-like humans	
Reward	**shift**	ρ=0.5	ρ=1	ρ=0.5	ρ=1
Label-based (all)	99.7 ± 0.5 (0.2)	80.4 ± 2.9 (11.5)	80.6 ± 1.3 (11.7)	83.4 ± 7.1 (5.8)	82.3 ± 5.2 (5.4)
Audit recal., 2% labels	**97.6** ± 3.5 (1.5)	81.7 ± 1.7 (10.6)	79.4 ± 2.3 (12.6)	74.5 ± 2.3 (16.2)	58.1 ± 9.0 (25.8)
Audit recal., level-9 challenge	88.7 ± 4.7 (7.0)	77.0 ± 3.9 (13.0)	72.6 ± 1.4 (17.2)	59.5 ± 1.6 (25.8)	38.8 ± 5.6 (38.3)
Score-only	92.4 ± 0.4 (4.8)	**83.6** ± 1.7 (8.8)	**82.7** ± 0.8 (9.4)	49.8 ± 0.2 (32.6)	17.5 ± 0.2 (52.0)
Posterior (EM share)	92.7 ± 1.5 (4.1)	67.5 ± 8.7 (4.3)	5.0 ± 2.4 (7.3)	48.2 ± 1.0 (32.8)	13.0 ± 3.3 (56.8)
Static NP threshold[a]	95.6 (2.9)	54.8 (3.0)	13.7 (3.0)	66.1 (22.7)	40.7 (39.3)
Static NP multi-threshold[a]	94.3 (2.3)	51.7 (2.3)	8.5 (2.3)	**76.8** (12.1)	**59.9** (21.1)
Static threshold[a]	90.3 (6.4)	59.2 (6.4)	28.4 (6.5)	54.2 (30.1)	22.5 (51.4)
Static multi-threshold[a]	78.0 (10.7)	58.4 (10.5)	39.5 (10.8)	50.8 (24.7)	26.3 (36.9)

[a] Untrained static baselines (deterministic, no seed SD).

Against covert bots, audit recalibration performs like the score-only reward (81.7 and 79.4, within 2–3.5 points), and without shift it is close to the label-based reward (97.6 against 92.4 for score-only). Its advantage appears when bot-like humans shift the score (Section IV-D): there it keeps far more humans than score-only (58.1 against 17.5 at $\rho = 1$), because the audit tells the reward what a score means in the current traffic. The challenge audit (Eqn. (5)), counting its own friction and lost humans, costs about 4 DI without shift and 7–10 DI on covert bots relative to score-only, because some audited humans fail or leave and its labels are noisy. Audit recalibration with confirmed labels is thus the best reward short of labelling every user; without labels, score-only is the next best choice.

3) Training Online Only When a Shift Is Detected

Online training is needed under covert bots but costs performance when nothing has changed (PPO: 92.4 always online against 99.8 frozen without drift). We therefore gate it with a label-free change detector: a one-sided CUSUM [57], $S_t = \max(0, S_{t-1} + z_t - 0.5)$, on a per-episode statistic of the users' first-chunk scores, standardised by a 50-episode burn-in, with the threshold set so that at most 5% of 300-episode no-drift runs alarm. We compare the Kolmogorov–Smirnov distance to the burn-in scores with the share of users in the middle band $[0.25, 0.6]$, where neither humans (near 0.1) nor known bots (above 0.8) fall but covert bots do. PPO is trained offline for 200 episodes, deployed frozen, and switched to score-only online training once the detector fires; covert bots arrive 50 episodes into deployment (Table 6).

Neither detector fired in any no-drift run, so the gated policies keep the frozen policy's 99.8 and 0.1 s per human. Under covert bots the mid-band detector fired 1–5 episodes after the attack in every run, and the gated policy matched or exceeded always-online training (85.1 and 82.0, against 83.8 and 82.0), since it adapts from the fully trained offline policy; the KS detector fired after 5–79 episodes, missed one run at $\rho = 0.5$, and recovered less (74.8 and 79.0). A frozen policy plus a cheap change detector thus combines the no-drift performance of offline training with the robustness of

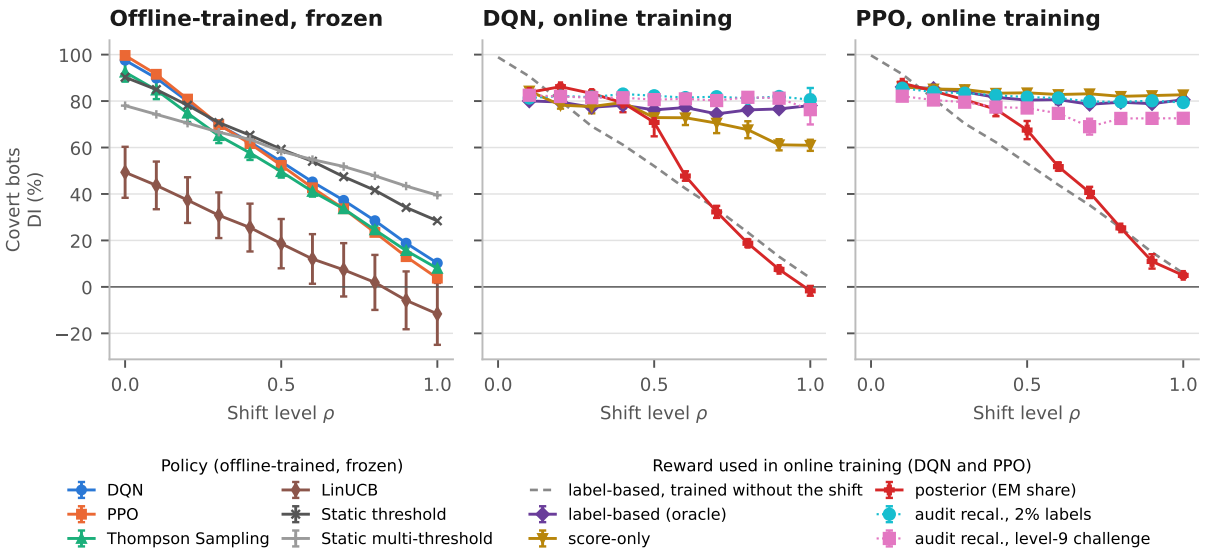


FIGURE 4. **DI on the evaluation block as covert bots replace a fraction ρ of the bots (mean $\pm$ SE over 5 training seeds). Left: offline-trained policies, frozen. Middle and right: online training of DQN and PPO (the two stable learners of Table 4) through the shift with three rewards, plus the two audit-recalibrated rewards of Table 5 (dotted); dashed: control, trained online with the label-based reward but without covert bots, then evaluated frozen.**

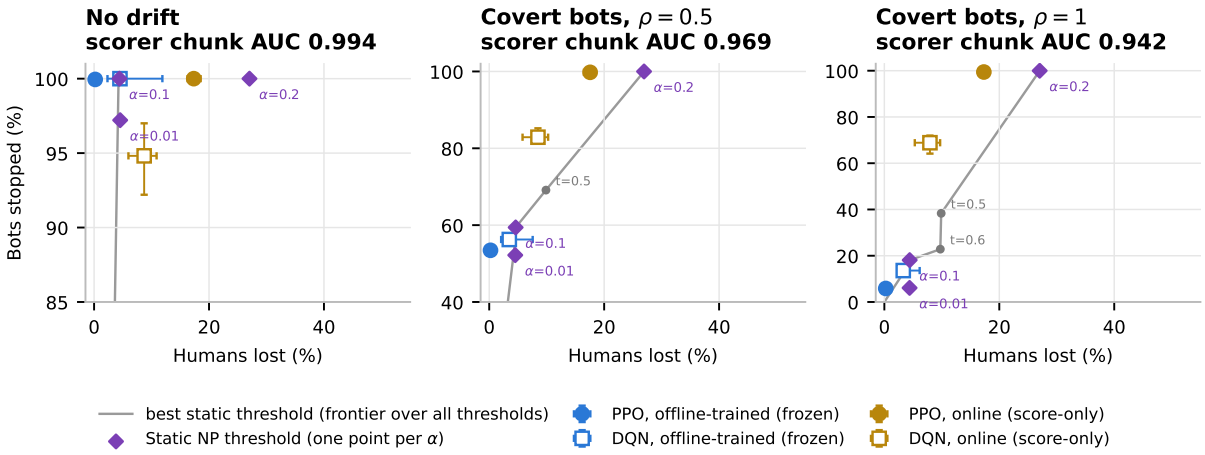


FIGURE 5. **Pareto-Frontier: Humans lost against bots stopped on the evaluation block. Each purple diamond is the static NP threshold at one human-challenge budget $\alpha \in \{0.01, 0.1, 0.2\}$; the grey line joins the non-dominated points over every single threshold tested (NP budgets 0.01–0.3 and raw score thresholds 0.1–0.9), the best trade-off a static threshold reaches. Each RL marker is the mean over the five false-positive penalties, with bars spanning the range over penalties. Better points lie towards the top left.**

online training. Because the detector reacts to any change in the score distribution, an alarm should trigger adaptation and operator review rather than be read as proof of an attack.

4) Covert Bots That Destroy the Ranking

In a stronger attack, a fraction ρ of bots *replay* recorded human mouse trajectories from the evaluation block, so that at $\rho = 1$ their score distribution equals the humans' (session AUC 0.75 at $\rho = 0.5$, 0.50 at $\rho = 1$). Every policy, frozen or trained online, static or learned, then degrades linearly to DI ≈ 0 at $\rho = 1$ (PPO 0.2, DQN -0.1, best static 3.8); even online training with all labels reaches only 22 DI for DQN, by challenging every user. This is a limit of the information available, not of the learner: when the attacker reproduces the very features the detector uses, no decision rule built on them can separate the classes, and defending against replay requires signals the attacker has not copied, such as challenge-response behaviour or device and network features.

5) Robustness to Bot-Like Humans

Legitimate users can also drift towards the bot side of the score, for example with unfamiliar input devices, remote desktops or assistive technology. As a false-positive check, a fraction ρ of the evaluation humans is replaced by Balabit users [42], recorded over remote desktop, whom the scorer

TABLE 6. Gated Online Training. DI on the Evaluation Block, Mean ± SD over 5 Training Seeds, with Friction (Seconds of Challenge per Human) in Parentheses

Deployment policy	No drift	Covert bots ρ=0.5	Covert bots ρ=1	Detection delay[a]
Always frozen	**99.8** ± 0.5 (0.1)	52.9 ± 1.2 (0.2)	4.6 ± 1.7 (0.2)	—
Always online (score-only)	92.4 ± 0.5 (4.8)	83.8 ± 2.0 (8.5)	82.0 ± 2.3 (9.4)	—
Gated, mid-band CUSUM	**99.8** ± 0.5 (0.1)	**85.1** ± 1.2 (8.0)	82.0 ± 3.1 (9.0)	**4 / 1**
Gated, KS CUSUM	**99.8** ± 0.5 (0.1)	74.8 ± 14.5 (5.2)	79.0 ± 6.5 (8.1)	77[b] / 6
Ungated audit baseline	98.3 ± 3.0 (1.1)	83.3 ± 2.1 (9.5)	**82.6** ± 1.0 (9.8)	—

[a]Median number of deployment episodes from the arrival of the covert bots to the alarm, at $\rho = 0.5$ / $\rho = 1$, over 5 seeds; no detector raised an alarm in any no-drift run. [b]One of the five runs never raised an alarm and is counted as the longest delay.

gives a mean $P(\text{bot})$ of 0.45. Frozen DQN and PPO keep DI at 57.9 and 56.4 at $\rho = 1$, above the NP threshold (40.7) and the multi-threshold policy (26.3), because they learned to challenge rather than block. The NP multi-threshold does best (59.9), since drifted humans that cross its threshold mostly land in its lower bands, whose challenges humans usually pass (Table 5). Online training helps only with the label-based reward (PPO 82.3 at $\rho = 1$) or audit recalibration (58.1); the score-only reward, which reads the higher scores as evidence of bots, makes it worse (17.5).

V. DISCUSSION AND LIMITATIONS

A. What the Evidence Supports

With a reliable behavioural score, a learned controller keeps more humans than static thresholds while stopping the same bots: it spends challenges only where the score and the session history point to a bot. The score is the component that matters most (DI 16.8 without it). The Full-RL learners are stable across seeds while contextual bandits are not (10.1-28.3 SD), and the result holds under varying traffic and across reward and simulator constants. The scorer itself transfers to a published benchmark once fine-tuned.

B. Limitations

(1) *Deployment reward*: the most robust policies need class information during online training. Rewards from observable outcomes alone fail, and scorer-based rewards are only as good as the scorer. Labelling a random 2% of users and recalibrating the score with them recovers most of the label-based performance (Table 5). When no labels are available, the score-only reward is the fallback, robust to covert bots but not to bot-like humans. (2) *Ranking-destroying attacks*: when bots reproduce the features the scorer uses, no policy built on that score can separate them; new signals are needed. (3) *Usability is modelled, not measured*: pass rates and times come from published studies of other populations and human abandonment is modelled. (4) *Simulator*: no real CAPTCHA provider, no request-rate, CPU or latency model, and no adaptive or VLM-based attacker, although the drift study shows how policies respond when an unseen bot type arrives.

C. Future Work

A user study of the challenge ladder, abandonment measured at the moment a challenge is shown, integration with real CAPTCHA providers, a queueing model for server load, attackers that adapt to the learned policy, and label-efficient rewards that stay robust when the scorer drifts.

VI. CONCLUSION

We framed CAPTCHA orchestration as learning a mapping from a calibrated behavioural score and session context to an 11-level intervention ladder, and evaluated it in a simulator grounded in published CAPTCHA user studies, with a leakage-controlled data protocol. On held-out recordings PPO and DQN reach DI 99.8 and 97.8 against 95.5 for the best static policy, and without the score DQN falls to 16.8. The scorer-only PPO online model adapts to calibration shift due to covert bots, beating all static methods and matching labelled approaches. With online training only when dataset drift is detected, score-only PPO matches or exceeds all labelled approaches. We also shows the limitations of this approach: no reward computed from observable outcomes alone adapted to a ranking-destroying attack, and policies trained or adapted with scorer-derived rewards inherit the scorer's miscalibration. A practical system therefore needs a calibrated scorer, occasional ground-truth labels and calibration monitoring. With these, RL is a viable replacement for hand-tuned score thresholds.

ACKNOWLEDGMENT

AI tools were used for language editing and formatting. The authors thank the participants who provided mouse-movement data.

REFERENCES

[1] Imperva, "2025 Imperva Bad Bot Report," Thales Group, 2025.
[2] E. Ferrara, O. Varol, C. Davis, F. Menczer, and A. Flammini, "The rise of social bots," *Commun. ACM*, vol. 59, no. 7, pp. 96–104, 2016.
[3] M. H. N. Ba, J. Bennett, M. Gallagher, and S. Bhunia, "A case study of credential stuffing attack: Canva data breach," in *Proc. Int. Conf. Comput. Sci. Comput. Intell. (CSCI)*, 2021.
[4] L. von Ahn, M. Blum, N. J. Hopper, and J. Langford, "CAPTCHA: Using hard AI problems for security," in *Advances in Cryptology – EUROCRYPT 2003*, pp. 294–311.
[5] M. Guerar, L. Verderame, M. Migliardi, F. Palmieri, and A. Merlo, "Gotta CAPTCHA 'em all: A survey of 20 years of the human-or-computer dilemma," *ACM Comput. Surv.*, vol. 54, no. 9, Art. 192, 2021.
[6] E. Bursztein, S. Bethard, C. Fabry, J. C. Mitchell, and D. Jurafsky, "How good are humans at solving CAPTCHAs? A large scale evaluation," in *Proc. IEEE Symp. Security and Privacy*, 2010, pp. 399–413, doi: 10.1109/SP.2010.31.
[7] A. Searles, Y. Nakatsuka, E. Ozturk, A. Paverd, G. Tsudik, and A. Enkoji, "An empirical study & evaluation of modern CAPTCHAs," in *Proc. 32nd USENIX Security Symp.*, 2023, pp. 3081–3097.
[8] M. Qi *et al.*, "VIPER Strike: Defeating visual reasoning CAPTCHAs via structured vision-language inference," in *Proc. 35th USENIX Security Symp.*, 2026.
[9] Z. Wu *et al.*, "MCA-Bench: A multimodal benchmark for evaluating CAPTCHA robustness against VLM-based attacks," in *Proc. AAAI Conf. Artif. Intell.*, vol. 40, no. 44, 2026.
[10] A. Plesner, T. Vontobel, and R. Wattenhofer, "Breaking reCAPTCHAv2," in *Proc. IEEE COMPSAC*, 2024.

[11] Google reCAPTCHA. [Online]. Available: https://www.google.com/recaptcha/about

[12] M. Sepczuk and Z. Kotulski, "A new risk-based authentication management model oriented on user's experience," *Comput. Secur.*, vol. 73, pp. 17–33, 2018.

[13] T. S. Sethi, M. Kantardzic, L. Lyu, and J. Chen, "A dynamic-adversarial mining approach to the security of machine learning," *WIREs Data Mining Knowl. Discov.*, vol. 8, 2018.

[14] S. Sivakorn, J. Polakis, and A. D. Keromytis, "I'm not a human: Breaking the Google reCAPTCHA," in *Black Hat Asia*, 2016.

[15] W. A. Yousef, I. Traoré, and W. Briguglio, "Classifier calibration: With application to threat scores in cybersecurity," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 3, pp. 1994–2010, 2023.

[16] Google Cloud Armor. [Online]. Available: https://cloud.google.com/security/products/armor

[17] M. Umar and S. Ayyub, "A review on evolution of various CAPTCHA in the field of web security," *IJRASET*, 2022.

[18] C. Iliou, T. Kostoulas, T. Tsikrika, V. Katos, S. Vrochidis, and I. Kompatsiaris, "Detection of advanced web bots by combining web logs with mouse behavioural biometrics," *Digit. Threats: Res. Pract.*, vol. 2, no. 3, Art. 24, 2021, doi: 10.1145/3447815.

[19] J. Yan and A. S. El Ahmad, "Usability of CAPTCHAs or usability issues in CAPTCHA design," in *Proc. SOUPS*, 2008.

[20] E. Bursztein, J. Aigrain, A. Moscicki, and J. C. Mitchell, "The end is nigh: Generic solving of text-based CAPTCHAs," in *Proc. USENIX WOOT*, 2014.

[21] I. Akrout, A. Feriani, and M. Akrout, "Hacking Google reCAPTCHA v3 using reinforcement learning," arXiv:1903.01003, 2019.

[22] A. Acien, A. Morales, J. Fierrez, and R. Vera-Rodriguez, "BeCAPTCHA-Mouse: Synthetic mouse trajectories and improved bot detection," *Pattern Recognit.*, vol. 127, 2022.

[23] S. Khan, C. Devlen, M. Manno, and D. Hou, "Mouse dynamics behavioral biometrics: A survey," *ACM Comput. Surv.*, vol. 56, no. 6, pp. 1–33, 2024, doi: 10.1145/3640311.

[24] C. Shen, Z. Cai, X. Guan, Y. Du, and R. A. Maxion, "User authentication through mouse dynamics," *IEEE Trans. Inf. Forensics Security*, vol. 8, no. 1, pp. 16–30, 2013.

[25] M. Takeda, T. Sato, H. Saito, H. Iwasaki, I. Nambu, and Y. Wada, "Explanation of Fitts' law in reaching movement based on human arm dynamics," *Sci. Rep.*, vol. 9, Art. 19804, 2019.

[26] S. Adarsh, P. Venkateshan, and P. A. Suresh, "Emulating human-like mouse movement using Bézier curves and behavioural models for advanced web automation," *IJIRT*, vol. 12, no. 1, 2025.

[27] M. Antal, K. Buza, and N. Fejer, "SapiAgent: A bot based on deep learning to generate human-like mouse trajectories," *IEEE Access*, vol. 9, 2021.

[28] A. Acien, A. Morales, J. Fierrez, R. Vera-Rodriguez, and O. Delgado-Mohatar, "BeCAPTCHA: Behavioral bot detection using touchscreen and mobile sensors benchmarked on HuMIdb," *Eng. Appl. Artif. Intell.*, vol. 98, 2021.

[29] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[30] Z. He, Y. Song, S. Zhou, and Z. Cai, "Mouse dynamics based bot detection using sequence learning," in *Proc. Chinese Conf. Biometric Recognit. (CCBR)*, 2021, pp. 49–56.

[31] J. Chandra *et al.*, "Aura-CAPTCHA: A reinforcement learning and GAN-enhanced multi-modal CAPTCHA system," arXiv:2508.14976, 2025.

[32] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proc. 19th Int. Conf. World Wide Web*, 2010, pp. 661–670, doi: 10.1145/1772690.1772758.

[33] D. J. Russo, B. Van Roy, A. Kazerouni, I. Osband, and Z. Wen, "A tutorial on Thompson sampling," *Found. Trends Mach. Learn.*, vol. 11, no. 1, pp. 1–96, 2018, doi: 10.1561/2200000070.

[34] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.

[35] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[36] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[37] Y. Cui, X. Han, J. Chen, X. Zhang, J. Yang, and X. Zhang, "FraudGNN-RL: A graph neural network with reinforcement learning for adaptive financial fraud detection," *IEEE Open J. Comput. Soc.*, 2025.

[38] P. K. Mvula, P. Branco, G.-V. Jourdan, and H. L. Viktor, "A systematic literature review of cyber-security data repositories and performance assessment metrics for semi-supervised learning," *Discover Data*, 2023.

[39] D. Arp *et al.*, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symp.*, 2022.

[40] D. H. Wolpert, "Stacked generalization," *Neural Netw.*, vol. 5, no. 2, pp. 241–259, 1992, doi: 10.1016/S0893-6080(05)80023-1.

[41] V. Chernozhukov, D. Chetverikov, M. Demirer, E. Duflo, C. Hansen, W. Newey, and J. Robins, "Double/debiased machine learning for treatment and structural parameters," *Econometrics J.*, vol. 21, no. 1, pp. C1–C68, 2018.

[42] Á. Fülöp, L. Kovács, T. Kurics, and E. Windhager-Pokol, Balabit Mouse Dynamics Challenge data set, 2016. [Online]. Available: https://github.com/balabit/Mouse-Dynamics-Challenge

[43] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML*, 2017, pp. 1321–1330.

[44] C. Scott and R. Nowak, "A Neyman–Pearson approach to statistical learning," *IEEE Trans. Inf. Theory*, vol. 51, no. 11, pp. 3806–3819, 2005, doi: 10.1109/TIT.2005.856955.

[45] X. Tong, Y. Feng, and J. J. Li, "Neyman–Pearson classification algorithms and NP receiver operating characteristics," *Sci. Adv.*, vol. 4, no. 2, eaao1659, 2018, doi: 10.1126/sciadv.aao1659.

[46] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*, A. J. Smola, P. Bartlett, B. Schölkopf, and D. Schuurmans, Eds. Cambridge, MA, USA: MIT Press, 2000, pp. 61–74.

[47] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure," *Neural Comput.*, vol. 14, no. 1, pp. 21–41, 2002, doi: 10.1162/089976602753284446.

[48] B. Zadrozny and C. Elkan, "Obtaining calibrated probability estimates from decision trees and naive Bayesian classifiers," in *Proc. ICML*, 2001, pp. 609–616.

[49] G. Forman, "Quantifying counts and costs via classification," *Data Min. Knowl. Discov.*, vol. 17, no. 2, pp. 164–206, 2008, doi: 10.1007/s10618-008-0097-y.

[50] C. Riquelme, G. Tucker, and J. Snoek, "Deep Bayesian bandits showdown: An empirical comparison of Bayesian deep networks for Thompson sampling," in *Proc. ICLR*, 2018.

[51] J. Hong, B. Kveton, M. Zaheer, M. Ghavamzadeh, and C. Boutilier, "Thompson sampling with a mixture prior," in *Proc. AISTATS*, PMLR vol. 151, 2022, pp. 7565–7586.

[52] A. Birnbaum, "Some latent trait models and their use in inferring an examinee's ability," in F. M. Lord and M. R. Novick, *Statistical Theories of Mental Test Scores*. Reading, MA: Addison-Wesley, 1968, pp. 397–479.

[53] S. E. Embretson and S. P. Reise, *Item Response Theory for Psychologists*. Mahwah, NJ: Lawrence Erlbaum, 2000.

[54] F. B. Baker, *The Basics of Item Response Theory*, 2nd ed. College Park, MD: ERIC Clearinghouse on Assessment and Evaluation, 2001.

[55] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. ICLR*, 2016.

[56] P. Christen, D. J. Hand, and N. Kirielle, "A review of the F-measure: Its history, properties, criticism, and alternatives," *ACM Comput. Surv.*, vol. 56, no. 3, Art. 73, 2024.

[57] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1–2, pp. 100–115, 1954, doi: 10.1093/biomet/41.1-2.100.